

Technical Evaluation Report

JAVA Assessment

Candidate:	nitin.bharti@irissoftware.com
Email:	nitin.bharti@irissoftware.com
Tech Stack:	JAVA
Completed:	August 26, 2026 at 05:47 PM
Report Generated:	August 26, 2026 at 05:48 PM

Overall Assessment

87.7 / 100

Evaluation Duration

Metric	Value
Actual Time Taken	50.1 minutes
Planned Duration	65 minutes
Time Efficiency	77.1% of planned time ✓

Category Scores

Category	Score
Terminologies	96.0/100
Semantics & Kafka connect	92.3/100
Kafka Producer & Consumer	71.2/100
Fundamentals & Architecture	83.3/100
Distributed messaging system	95.8/100

Communication Assessment

Metric	Score
Confidence Level	7.1/10
English Proficiency	7.0/10
Clarity Score	6.7/10

Strengths

- Recognized the hierarchy of topic -> partitions
- Answered both parts of the question: what it is and why it is needed
- Accurately mentions `sendOffsetsToTransaction` rather than normal consumer offset commit
- Answered both the comparison and the usage-choice parts of the question
- Strong framing of why delivery semantics matter in distributed systems

Areas for Improvement

- Use `record.offset()` and store `new OffsetAndMetadata(record.offset() + 1)` as the map value
- Mention the abort path explicitly: if transformation or produce fails, `abortTransaction()` must be called so neither output nor offsets are committed.
- Correct the offset explanation: offsets are not where the publisher decides to push the next message; Kafka assigns offsets to records in a partition, and consumers track them to know what they have read.
- Clarify that offsets are independent for each partition. Partition 0, partition 1, and partition 2 each have their own offset sequence starting from the first record written there.
- Clarify that lag is usually measured per partition and then viewed at the consumer-group level.

Detailed Question Feedback

Q1: Please explain your most recent project in detail. Describe the technologies used, your role and responsibilities, the challenges you faced, and how you overcame them. Also explain the architecture and any significant technical decisions you made.

Category: Project Experience | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

So in the last project I was associated with one of the giant bank which is a T bank and under that I was involved in the Dynamics project So retail banking is one of the unit of that project is basically been moving out from the project and they are having They are providing the support to that Stand alone unit so that one once it is moved out so whatever Business that is being run in today's scenario right today's date That is also being continued in the same manner. So whatever is how same downstream application I've been coming connected whatever the force are

being generated whatever of the Services that it being running so those we have to make a stand alone So I was working in that project as a team lead. So my role and responsibility responsibility was to daily Conduct the scrum meeting so that just want to understand what All the team members have done yesterday and what we are having What is the plan for today? And if some if there's someone who is facing some challenges so if some challenges are there so we Discuss that and try to solve it as soon as possible So in the project basically because that is being run on this scrum basis So we are having a sprint sprint planning in which we have decided to Come up with the task which we need to pick for the particular sprint which is of three weeks and once we the sprint is Over so we are having a retrospective best friend call which is basically to identify what all gaps we have but task are basically not been Complete which is been in this friend planning and we will move it in the next Cycle and once the sprint is completed. So we on the over the weekend we deploy the changes in the production so that it will be available for the client to See those changes in the production one word There are multiple challenges that we have faced in the project. So in terms of I mean because I started as an in the in the very beginning phase so the Coding centers and the code maintain and may maintain standard was not being followed So I worked on those challenges and I prepared Confluence over that to whatever things we have to follow like for example, if we are committing some message, right? Some changes so we have to pass the proper commit message and if a creating branch so how we can create branch whether it's a feature release or any bug branch and Apart from that there were challenges in the module like I Notification was not being picked properly for the portal so we come up with a plan to when prepare the Magnet service that will handle the notification and it will be Available on the screen the top two notification, which will show the system is now so it will be simple Project architecture is basically we are having the binomics portal which is basically on the angular side and that is being used by the customer and the client basically use that Clients basically use that Portal to interact with the system and we are having a back-end support services Which is my on the micro service framework, which are being run and it is one of the services This is going to put on which is a bad one for the service and it is being used for example the recently blue tabs and the Admin it used SSO to check the Robin authentication using tokens and Likewise we are having multiple Other services and the job which are being run using the autosys so we use autosys to schedule the job, which is based in daily Daily monthly depending on the scenario Domain I already explained that it is a bank domain and it's one of the retail unit which is being moved off from the city bank So we are supporting that project tools and technology than to use. So we are using the technology flight in the Java We are using the technology like Java 17 along with that we are using the spring boot Kafka we are using and we were using the Autosys the light speed is for building and hardness for deployment and may one for building the Java code packaging the code and

Good: Good overview of the project and technologies used, Correctly identified role as team lead

Improve: Provide a clear explanation of the project architecture and significant technical decisions made, Be more accurate with technology versions (Java, Spring Boot)

Detailed Feedback:

The candidate's answer provides a good overview of their recent project, but lacks depth and technical accuracy in some areas. They correctly identified the technologies used (Java, Spring Boot, Kafka, Autosys) and described their role as team lead. However, they failed to provide a clear explanation of the project architecture and significant technical decisions made. The candidate also incorrectly stated that they were using Java 17 along with Spring Boot, when in fact they mentioned using Spring Boot without specifying the version of Java. Additionally, they missed explaining how the notification system was improved.

Q2: What does "offset" refer to in Kafka?

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

A unique, sequential identifier for a record's position within a partition

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'A unique, sequential identifier for a record's position within a partition' is correct.

Q3: Explain the difference between Standalone and Distributed mode in Kafka Connect and when you'd choose each.

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

The difference between standalone and distributed mode at KafkaConnect is basically the standalone mode runs KafkaConnect as a single GBM process on a single machine. It manages connector configuration via local file and stores its offset state locally. Whereas the distributed mode runs KafkaConnect across a cluster of multiple worker nodes using a shared group ID, it manages configuration dynamic via REST API and leverages internal Kafka topic to store metadata offset and configurations. So we need to choose the standalone mode when we are having a local development and testing, quickly testing the custom connector on your laptop, strict locality consummate, streaming log files from a specific host where the agent must physically live on that machine and one off batch task, simple utility pipelines where high availability is not necessary. And when we need to use distributed mode, distributed mode we have to use in the production environment where we need high availability, automatic failover and give you down time, high throughput and dynamic scale, you need to add or remove worker to balance large process volumes without restarting the cluster and the centralized management, you want to manage tasks dynamically across the department via automated CI CD pipeline calling the REST API. In a standalone mode connectors runs connector on as a single GBM process configuration are driven by local properties files and state offset and saved for local disk. It has no built-in fault tolerance. I recommend this principle primarily for local development, quick testing or edge case scenario where you must guarantee execution on a specific machine like pulling local system lock files. And in the distribution mode, multiple workers not joined together under a common group ID for 2.0 default tolerance cluster. They coordinate using internal copper topic to store configuration and state data and the connectors are managed automatically using the REST API. If the node dies, the cluster automatically rebalance task on the healthy node. This is the standard choice for all production workloads because of high availability and the horizontal scalability.

Score: 88/100

Good: Answered both the comparison and the usage-choice parts of the question, Demonstrated solid operational understanding of Kafka Connect deployment modes, Correctly connected distributed mode with HA, scaling, and REST-based centralized management, Provided practical scenarios for when standalone is appropriate

Improve: Use correct terminology consistently: say worker or JVM process instead of 'GBM process', Replace garbled/non-standard phrases like 'internal copper topic' with 'internal Kafka topics', State more precisely that distributed mode stores config, offsets, and status in Kafka topics shared by the worker cluster, Reduce repetition and present the comparison more cleanly and structurally

Detailed Feedback:

This is a strong answer that directly addresses both parts of the question: the difference between Kafka Connect standalone vs distributed mode, and when to choose each. The candidate correctly explained that standalone runs as a single worker/process on one machine, uses local configuration files, and is suited for development/testing or simple single-node use cases. They also correctly explained that distributed mode uses multiple workers in a cluster, stores config/offset/status in Kafka internal topics, supports REST-based management, and is the right choice for production due to scalability and fault tolerance. Technically, the core concepts are accurate and the candidate demonstrates real understanding rather than just naming keywords. They also covered operational implications such as failover, rebalancing, and dynamic scaling, which are relevant to the choice between the two modes. There are a few issues preventing an even higher score. Some phrasing appears to be transcription noise or misstatements: "GBM process" is not Kafka terminology and should be understood as a worker/JVM process; "internal copper topic" is clearly a transcription error for internal Kafka topics; "2.0 default tolerance cluster" is garbled and not meaningful as stated. Also, one nuance is that saying standalone stores state locally is broadly acceptable in interview context, but the more precise distinction is that standalone uses local files for configuration and does not use Kafka for distributed coordination, while distributed persists config/offset/status in Kafka internal topics shared by the cluster. The answer was also somewhat repetitive. Overall, this is a very good answer: accurate, complete for the scope asked, and clearly production-aware.

Q4: Explain what a consumer group rebalance is and its impact on processing.

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

The consumer group rebalance is basically a step or the process if some consumer is being moving in or moving out within that group. So some rebalance algorithm will be running, the process will be running, that will take some downtime and again once that being done it is the new consumer being added or no depending on that rebalance is happening after the node is down or it is being added. So during this rebalancing for a shorter while, for a shorter while the consumer would not be available to process and that would impact the processing or the consumption of the messages.

Score: 66/100

Good: Answered both parts of the question: what rebalance is and its impact, Correctly identified join/leave events as rebalance triggers, Correctly recognized that processing is temporarily interrupted during rebalance

Improve: Explicitly define rebalance as redistribution of topic partitions across consumers in the same consumer group, Use more precise terminology than "downtime"; the key impact is temporary pause in message consumption during partition revocation/reassignment, Clarify that rebalance can also happen when a consumer goes down or is considered dead by the group coordinator, not only when one is intentionally added or removed

Detailed Feedback:

The candidate demonstrates a basic correct understanding of a consumer group rebalance and addresses the impact on processing, which is exactly what the question asked. They correctly convey that rebalance happens when consumers join or leave the group, and that during the rebalance there is a temporary pause where consumers are not available to process messages, which impacts consumption. However, the answer is only moderately complete. A stronger explanation should clearly state that a rebalance is the redistribution/reassignment of topic partitions among consumers in the same Kafka consumer group when group membership changes (or similar triggering events occur). The candidate refers to consumers moving in or out and mentions a rebalance algorithm, but does not explicitly explain partition reassignment, which is the core technical meaning of a rebalance. They also mention "downtime," which is directionally understandable, but imprecise; the more accurate statement is that consumption is temporarily paused while partitions are revoked and reassigned, not necessarily full system downtime. So overall: technically meaningful, mostly correct at a high level, but missing the central concept of partition redistribution and using somewhat vague wording.

Q5: Explain the relationship between topics, partitions, and offsets in Kafka.

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

So, if you talk about the hierarchy, the topic is on the other where the first level inside topics that having partition method partition we are having opposite. So, topic is nothing but the channel through which the message is not being pushed from the publisher and which is being created by the consumer also. So, how the message is being pushes or stored inside a topic is being handled by the partition. So, the message are basically the topic is having more than one or more than one partition to store the messages locally in that topic basically. So, inside a partition we are having a reference number value which refers to the point where the publisher need to push the message or the next message which is being called outside. Suppose I am having a topic with name my topic inside topic I am having three partition, partition 0, partition 1 and partition 2. So, once the message is being pushed, so the offset is by default 0 for all three partitions because none of the messages are available, when the top message is being published in the topic. So, the offset of the topic partition 1 or 0 become offset is equal to 1 because one message is pushed because one message is pushed inside the partition, so this is offset. We can have multiple partition depending on the scenario.

Score: 48/100

Good: Recognized the hierarchy of topic -> partitions, Understood that a topic can have multiple partitions, Attempted to explain offsets with a concrete partition example, Provided some meaningful Kafka terminology rather than purely vague statements

Improve: Define the relationship more precisely: a topic is a logical stream, partitions are the subdivisions where records are stored, and offsets are the sequential positions of records within each partition., Correct the offset explanation: offsets are not where the publisher decides to push the next message; Kafka assigns offsets to records in a partition, and consumers track them to know what they have read., Clarify that offsets are independent for each partition. Partition 0, partition 1, and partition 2 each have their own offset sequence starting from the first record written there., Avoid misleading statements such as the topic being 'created by the consumer also' unless you explain the actual admin/client behavior, because it does not answer the relationship asked.

Detailed Feedback:

The candidate shows partial understanding of Kafka's core storage model, but the explanation is technically shaky and incomplete for the question asked. The question asked for the relationship between topics, partitions, and offsets. The candidate did identify the hierarchy at a basic level: a topic contains partitions, and messages are stored in partitions. They also attempted to explain offsets as a numeric reference within a partition. That is the main substance of the answer. However, there are important inaccuracies and missing clarifications. A topic is not well defined here: calling it a channel is acceptable at a high level, but saying it is 'created by the consumer also' is misleading in this context and

does not explain the relationship asked. The biggest issue is the explanation of offsets. In Kafka, offsets are per-partition sequential identifiers for records. They identify the position of a record within a partition, and consumers track offsets to know what has been read. The candidate instead described offset as 'the point where the publisher need to push the message or the next message,' which is not the right framing. Producers append records to partitions; offsets are assigned by Kafka to records in that partition, not used by the publisher as a pointer to where to push next. There is also confusion in the example. Saying offsets are 'by default 0 for all three partitions because none of the messages are available' is imprecise. More accurately, the first record written to a partition typically gets offset 0, the next gets 1, and so on, independently per partition. Offsets do not advance across the whole topic; each partition has its own offset sequence. The candidate did not clearly state that independence, which is a key part of the relationship. Overall, this is not an empty or irrelevant answer; it contains meaningful Kafka concepts. But it falls below average because the core relationship is only partially explained and the offset concept is not accurately described enough.

Q6: What is the main advantage of decoupling producers and consumers in a distributed messaging system?

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

Producers and consumers can operate independently at different rates without direct coupling

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Producers and consumers can operate independently at different rates without direct coupling' is correct.

Q7: What is a "rebalance" in the context of consumer groups?

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: hard | Type: OBJECTIVE

Candidate's Verbal Answer:

The process of redistributing partition ownership among consumers in a group

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'The process of redistributing partition ownership among consumers in a group' is correct.

Q8: What is Kafka Connect, and why would a team use it instead of writing custom producer/consumer code for integrations?

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

Kafka Connect is a free distributed framework built into a patch designed specifically to stream data between Kafka and external service systems like database, S3, Elasticsearch or Data Warehouse without writing the application code. So it uses declarative configuration like JSON or the properties file to run pre-built source connector or sync connector. So why I use Kafka Connect versus the custom producer consumer code. So when we decide whether to use Kafka Connect or write customer using the Java or Python connector. So we can think of like a Kafka Connect as a managed infrastructure level integration framework and custom code as application level logic. Here is why would you choose Kafka Connect. So out of the boss ecosystem, Kafka Connect leverage a hundred of pre-tested enterprise great connectors. Whereas a custom producer code, you must manually code the network drive, batching, authentication and protocols. All told in offset, built-in management, it's automated tracks, offset and handles, no crashes by revealing tasks to health issues. This is in the Kafka Connect but in the custom producer, you must manually handle offset commits, failover, replies and thread crashes. In the scale and parallelism, so in a Kafka Connect scale out horizontally across a worker cluster by updating configuration connect, split tasks across partition automatically but in the consumer producer, you must write a custom thread management and container orchestrations and work partition assignment logic. So when we need to use, choose Kafka Connect, so Kafka Connect we need to use to when we are moving data to or from our data source like RDB as no SQL, objects,

storage, search index, where the primary job is data integration and when to you when we can write a custom code, when you need to process incoming message with the complex business logic. Example, executing a domain role called external web service API's per message.

Score: 82/100

Good: Answered both the definition and comparison parts of the question, Showed real understanding of operational benefits like scaling, offsets, and fault tolerance, Correctly identified the best-fit use case for Kafka Connect versus custom code, Used relevant examples of external systems such as databases, S3, Elasticsearch, and data warehouses

Improve: Define Kafka Connect more precisely as Apache Kafka's connector framework for importing/exporting data via source and sink connectors., Use more accurate terminology: custom producer/consumer code is not the same thing as a "Java or Python connector.", State more explicitly that teams use Kafka Connect to avoid building and maintaining integration plumbing such as retries, scaling, offset/state handling, and connector lifecycle management.

Detailed Feedback:

The candidate gave a solid and mostly accurate answer to both parts of the question: what Kafka Connect is, and why a team would use it instead of writing custom producer/consumer integration code. They correctly described Kafka Connect as a framework for moving data between Kafka and external systems, highlighted that it uses prebuilt source and sink connectors with declarative configuration, and contrasted it with custom application code. They also correctly explained several practical reasons teams choose it: reusable connectors, built-in offset handling, fault tolerance/restarts, and horizontal scaling across workers/tasks. They additionally made an important distinction that Kafka Connect is best for infrastructure/data integration use cases, while custom code is better when complex business logic is required. There are some technical rough edges and imprecise phrasing. "Built into Apache" was likely intended, but Kafka Connect is part of the Kafka ecosystem/platform rather than something you would describe as a generic "free distributed framework built into a patch." Also, the answer blurred producer/consumer code with "Java or Python connector," which is imprecise terminology. A stronger answer would have explicitly said Kafka Connect is a framework for running connectors in distributed or standalone mode, with source connectors importing data into Kafka and sink connectors exporting data from Kafka. It also would have more clearly emphasized reduced operational burden and standardized integration management. Overall, though, the substance is good: the candidate understands the purpose of Kafka Connect and the main reasons to prefer it over custom integration code.

Q9: What is a potential consequence of setting "max.poll.interval.ms" too low for a slow-processing consumer?

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: hard | Type: OBJECTIVE

Candidate's Verbal Answer:

The consumer may be considered dead and removed from the group, triggering a rebalance

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'The consumer may be considered dead and removed from the group, triggering a rebalance' is correct.

Q10: Why is Kafka often described as "pub-sub with a log"?

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

In pub-sub system like Kafka, the offset X as a bookmark representing a consumer exit position in an ordered lock, the lock mechanism achieved via partition assignment inside consumer groups ensure that only one consumer process is a given partition at any time, combining offset while exclusive partition ownership allows distributed consumer to achieve high throughput, failover resilience and deterministic offset recovery without message to be issued.

Score: 52/100

Good: Demonstrates genuine familiarity with Kafka consumer mechanics, Correctly references offsets as a bookmark/position mechanism, Understands partition ownership within consumer groups, Mentions operational benefits like failover and offset recovery

Improve: Answer the phrase directly: Kafka is called 'pub-sub with a log' because it supports publish-subscribe messaging, but stores messages in an ordered persistent log that consumers read by offset., Explicitly mention that consumers can read independently and replay messages because records remain in the log for a retention period., Separate the core concept from consumer-group details; partition ownership is secondary to the main explanation.

Detailed Feedback:

The candidate shows some real Kafka knowledge, but the answer does not directly and clearly answer the specific question: why Kafka is described as 'pub-sub with a log'. Interpreting the transcription, they are saying that Kafka uses offsets as bookmarks in an ordered log, and that partition assignment in consumer groups gives exclusive ownership of a partition to one consumer at a time. Those points are technically relevant and mostly correct: Kafka does have an ordered append-only log per partition, consumers track offsets, and consumer groups coordinate partition ownership. However, the answer is incomplete for the actual question. The key idea behind 'pub-sub with a log' is that Kafka combines publish-subscribe messaging semantics with a persistent ordered log: producers publish records to topics, multiple consumers can subscribe, and messages are retained in the log so consumers can read at their own pace, replay from offsets, and recover independently rather than messages being deleted immediately after delivery. The candidate touched the 'log' part through offsets and ordering, but did not clearly explain the 'pub-sub' part, nor the combination of the two. They also focused heavily on consumer-group partition ownership, which is related but not the core explanation of the phrase. There are also likely transcription artefacts such as 'ordered lock' meaning 'ordered log' and 'consumer exit position' meaning 'consumer's position'. I did not penalize those. But even after correcting for transcription, the answer remains only partial. It demonstrates understanding of offsets, partitions, and consumer groups, but misses the central conceptual explanation of the phrase being asked.

Q11: How does Kafka achieve horizontal scalability?

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

By distributing topic partitions across multiple brokers

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'By distributing topic partitions across multiple brokers' is correct.

Q12: Define "consumer lag" and explain its operational importance.

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

Consumer lag is the gap between the latest offset written to a Kafka partition by the producer and the last offset that is being committed by the consumer group. It directly measures how far behind a consumer is from a real-time data flow. Operationally consumer lag is the single most critical health metric for monitoring the real-time pipeline. A steadily increasing lag signal performance bottlenecks such as slow downstream database calls, thread crashes or sudden producer traffic surges. Monitoring lag helps prevent SLA violation and avoid data loss caused by the log retention expiration. It also serves as the primary metric for auto-skilling consumer instance for a down-to-match incoming message throughput.

Score: 91/100

Good: Correctly answered both the definition and operational importance, Used accurate Kafka offset-based reasoning, Showed practical operational understanding, not just a textbook definition, Connected lag to monitoring, throughput issues, and scaling decisions

Improve: Make the definition slightly more precise by referring to the latest partition offset (log end offset) versus the consumer group's current committed/consumed offset., Clarify that lag is usually measured per partition and then viewed at the consumer-group level.

Detailed Feedback:

This is a very strong answer. The candidate correctly defines consumer lag in Kafka terms as the difference between the latest produced offset in a partition and the offset the consumer group has committed/processed, which is the core concept the question asked for. They also clearly explain its operational importance: it indicates how far behind consumers are, helps detect bottlenecks or traffic spikes, is a key health metric for real-time pipelines, and matters for SLA adherence and retention-related risk. These are all technically relevant and accurate in the Java/Kafka ecosystem context. Minor caveat: the most precise definition is usually the difference between the partition's latest offset (log end offset) and the consumer group's current consumed/committed offset. In practice, saying 'last offset being committed by the consumer group' is acceptable, though committed offset is not always identical to the last message actually processed at a given instant depending on commit strategy. Also, 'data loss caused by log retention expiration' is directionally correct operationally: if lag grows beyond retention, consumers may miss older messages because they are no longer available. The final phrase 'auto-skilling' is clearly a transcription artifact for 'auto-scaling,' and 'for a down-to-match' appears to mean scaling consumers up/down to match throughput; this should not be penalized. Overall, the candidate answered both parts of the question well: definition and operational importance. The answer is accurate, complete for the scope asked, and demonstrates real operational understanding rather than rote memorization.

Q13: Explain the end-to-end steps involved in achieving exactly-once processing in a "consume-transform-produce" pipeline using Kafka transactions.

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

To achieve exactly one processing in a consumed-transform-produced pipeline, Kafka uses a two-phase commit protocol driven by a transaction coordinator, pairing an idempotent producer with a transactional.id static. In practice, the pipeline reads a batch of messages, transforms them and writes to the transformed record to output topic. Crucially, instead of committing input offset via the consumer, it calls sendOffsetsToTransaction method, binding both the output record and the input offset into a single atomic transaction when a commit transaction method is called. The coordinator writes commit marker to both the output topic and the consumer offset topic. Downstream consumer configured with the read committed will only read the output once those commit markers are written. If a crash occurs midway, the coordinator aborts the transaction, the uncommitted records are ignored and the next consumer that rereads from the last committed offset without data duplication or loss.

Score: 84/100

Good: Correctly explains the central EOS idea: output records and input offsets are committed atomically, Accurately mentions sendOffsetsToTransaction rather than normal consumer offset commit, Correctly identifies read_committed as necessary for downstream visibility semantics, Shows good understanding of failure handling and replay behavior, Uses mostly correct Kafka transactional terminology despite transcription issues

Improve: For an end-to-end answer, explicitly list the Java transactional steps: configure transactional.id, initTransactions(), beginTransaction(), process and produce, sendOffsetsToTransaction(...), then commitTransaction() or abortTransaction()., Mention the abort path explicitly: if transformation or produce fails, abortTransaction() must be called so neither output nor offsets are committed., Avoid loosely calling this a generic "two-phase commit protocol"; the more precise explanation is Kafka transaction coordination between produced records and committed offsets via the transaction coordinator.

Detailed Feedback:

This is a strong answer that correctly explains the core end-to-end mechanism for exactly-once processing in a Kafka consume-transform-produce pipeline. The candidate clearly understands that Kafka transactions combine produced output records and consumed offsets into a single atomic unit, and they correctly mention the transaction coordinator, idempotent producer, use of a transactional.id, sendOffsetsToTransaction, commitTransaction, and downstream consumers using read_committed. They also correctly describe the failure behavior at a high level: if the application crashes before commit, uncommitted output is not visible to read_committed consumers and the input will be re-read from the last committed offset. The answer is technically solid, but not fully complete for an end-to-end steps question. A more complete answer should explicitly include the operational sequence: configure producer with transactional.id, call initTransactions(), beginTransaction(), poll/process records, produce output, call sendOffsetsToTransaction(...) with the consumer group metadata, then commitTransaction() or abortTransaction() on failure. The candidate implied most of this flow but did not explicitly mention beginTransaction, abortTransaction, or initTransactions, which are important concrete steps in the Java/Kafka transactional API. Also, saying Kafka uses a "two-phase commit protocol" is directionally understandable, but that phrasing is not ideal here; Kafka transactions are coordinated by the transaction coordinator, but this is not typically described as a generic distributed 2PC explanation in application-level terms. Overall, the candidate demonstrates real understanding and answers the question well, but misses a few important

API-level steps that were part of the requested end-to-end explanation.

Q14: What is the purpose of the "linger.ms" producer config?

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

To wait a short time to batch more records before sending, improving throughput

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'To wait a short time to batch more records before sending, improving throughput' is correct.

Q15: What is the purpose of replication factor in Kafka?

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

To determine how many copies of each partition are maintained across brokers for fault tolerance

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'To determine how many copies of each partition are maintained across brokers for fault tolerance' is correct.

Q16: In distributed systems, what is the "in-sync replica" (ISR) set used for?

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: hard | Type: OBJECTIVE

Candidate's Verbal Answer:

Tracking replicas that are fully caught up with the leader for durability guarantees

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Tracking replicas that are fully caught up with the leader for durability guarantees' is correct.

Q17: What does "producer" mean in Kafka?

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

A client application that publishes records to topics

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'A client application that publishes records to topics' is correct.

Q18: What does "at-most-once" delivery semantics imply?

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

A message might be lost but is never processed more than once

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'A message might be lost but is never processed more than once' is correct.

Q19: Explain sticky vs cooperative sticky vs eager partition assignment strategies and their impact on rebalance disruption.

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: hard | Type: VERBAL

Candidate's Verbal Answer:

Partition assignment strategies dictate how our distributed topic partition among consumers particularly during a rebalance. Legacy assigners like range and round robin use the eager protocol causing stop the world pause where all the consumer drop their partition completely before resubmitting the assignment. Strictly assigner improve on this by minimizing partition movement between rebalance, preserving local cash threats for healthy consumers. Cooperative Strictly assigner is the modern production standard it uses incremental rebalance thing allowing unaffected consumer to continue processing data without pausing. Only the partition actually shifting between nodes are reworked and re-assigned effectively eliminating pipeline downtime during scale out or node failure event.

Score: 78/100

Good: Accurately describes eager rebalance disruption, Correctly identifies the main operational benefit of cooperative sticky, Shows practical understanding of partition movement minimization and reduced downtime, Good grasp of rebalance impact on running consumers

Improve: Explicitly distinguish StickyAssignor from CooperativeStickyAssignor: sticky minimizes movement but still uses eager rebalance, while cooperative sticky minimizes movement and uses incremental cooperative rebalance, State more clearly that eager is a rebalance protocol behavior, not just a property of older assignors, Make the comparison more structured: eager = full revoke, sticky = eager but movement-minimizing, cooperative sticky = movement-minimizing plus partial/incremental revoke

Detailed Feedback:

The candidate shows solid understanding of the main distinction between eager and cooperative rebalancing and correctly connects sticky assignment to minimizing partition movement. Interpreting the transcription, they are clearly referring to the StickyAssignor and CooperativeStickyAssignor. They correctly explain that older eager-style assignors revoke all partitions first, causing a stop-the-world pause, and that cooperative sticky uses incremental rebalancing so unaffected consumers can continue processing. They also correctly note the operational benefit of reduced disruption and better cache/locality preservation. However, the answer is not fully complete for the exact question asked: 'sticky vs cooperative sticky vs eager partition assignment strategies and their impact on rebalance disruption.' The candidate explains eager behavior and cooperative sticky behavior well, but does not clearly separate sticky from cooperative sticky as two distinct strategies/protocol behaviors. A complete answer should explicitly say that StickyAssignor is still a sticky balancing strategy but uses the eager rebalance protocol, so although it minimizes partition movement, it still requires all consumers to revoke partitions during rebalance. CooperativeStickyAssignor keeps the sticky goal but changes the rebalance protocol to cooperative/incremental, reducing disruption because only partitions that must move are revoked in phases. That distinction is the main missing piece. So technically this is a good answer with accurate substance, but it is somewhat incomplete because it blends sticky and cooperative sticky together instead of contrasting them explicitly.

Q20: What is Apache Kafka primarily designed for?

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

A distributed event streaming platform for high-throughput, fault-tolerant data pipelines

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'A distributed event streaming platform for high-throughput, fault-tolerant data pipelines' is correct.

Q21: What is the significance of "at least once", "at most once", and "exactly once" delivery semantics in distributed messaging?

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: easy | Type: VERBAL

Candidate's Verbal Answer:

In the distributed systems, network partition, hardware failures make absolute certainty impossible when sending data between nodes. Delivery semantics define the failure recovery contract between systems. So, at most once, proprietary, prioritize performance and zero duplication over durability, offsets are committed before processing, meaning a crash result in low strata is used for non-critical telemetry or metrics. At least once, prioritize durability. Data is processed before committing, ensuring no loss, but a network retry or crashes can cause duplication processing. This is a default industry standard typically paired with the idempotent consumer logic and downstream databases. Exactly once, semantic is provided into end correctness using the atomic transaction and the duplicated streams to guarantee that each record alters state prices precisely once, while it eliminates duplicate risk, it will introduce performance and let us try it off due to face committed overhead.

Score: 79/100

Good: Strong framing of why delivery semantics matter in distributed systems, Correctly identifies the core trade-offs among loss, duplication, durability, and performance, Good practical understanding of at least once and idempotent consumers, Recognizes that exactly once has operational/performance cost

Improve: Define each semantic more directly before moving into implementation details: at most once may lose messages, at least once may redeliver duplicates, exactly once ensures one effective processing outcome within supported transactional scope., Avoid making the explanation too Kafka-specific unless the question asks for implementation details; offset commits are one example, not the definition itself., Be more precise about exactly once: it is not a blanket guarantee everywhere, but typically requires coordinated transactional support and is limited to the systems participating in that guarantee.

Detailed Feedback:

The candidate demonstrates solid understanding of the three delivery semantics and, importantly, explains their significance in distributed systems as trade-offs under failure conditions. They correctly frames the topic around network partitions/crashes making perfect certainty difficult, which is a strong opening and directly relevant to why these semantics matter. Point-by-point: - "At most once": Mostly correct. They describe it as favoring performance and avoiding duplicates, with the risk of message loss if failure happens after acknowledging/committing but before processing. The phrase "offsets are committed before processing" is a Kafka-oriented explanation and is directionally correct for illustrating loss risk, though it is implementation-specific rather than the general definition. - "At least once": Correct. They explain that processing happens before commit/ack, reducing loss risk but allowing duplicates on retries/crashes. Mentioning idempotent consumers is a strong practical detail. - "Exactly once": Largely correct. They describe end-to-end correctness via atomic transactions and preventing duplicate effects so each record changes state once. They also correctly note the trade-off of higher overhead/latency/complexity. However, the wording is somewhat muddled, and exactly-once semantics are subtle: they are usually guaranteed only within specific system boundaries and require coordinated transactional support, not a universal guarantee across arbitrary downstream systems. Why this is not higher: the answer is good but not fully crisp. There are some transcription-garbled phrases (for example "proprietary" likely meant something else, and "low strata" likely meant "data loss"). Those are forgivable. More importantly, the candidate leans heavily on Kafka-style offset commit behavior instead of first giving the clean generic definitions. Also, exactly-once is described a bit too broadly; in practice it depends on transactional guarantees and scope. Overall, this is a good answer: technically meaningful, mostly accurate, and it addresses the significance of all three semantics through reliability/performance trade-offs under failure.

Q22: Define "leader epoch" and explain why it's needed.

Category: Terminologies | Question Category: Kafka_Upskilling | Difficulty: medium | Type: VERBAL

Candidate's Verbal Answer:

Leader Epoch in Kafka is a monotonically increasing counter-representing the generation of leadership for the partition. It was introduced to fix data loss and log divergence HKCs that occurred when reliant strictly on high watermarks for log transition during broker failure. When a broker recovers, instead of truncating its log to its local high watermark, it queries the current leader for the last offset return during its previous Leader Epoch. This ensures

followers only truncate uncommitted data that diverse during leadership transitions guarantee log consistency across replicas without silent message calls.

Score: 89/100

Good: Accurately defined leader epoch at a conceptual level, Answered both parts of the question: what it is and why it is needed, Demonstrated practical understanding of Kafka replica recovery behavior, Correctly connected leader epoch to avoiding divergence and data loss during failover, Showed understanding beyond a superficial textbook definition

Improve: State more explicitly that a leader epoch is incremented on each new leader election for a partition, Explain more directly that it helps replicas determine the correct log end offset for a prior leadership era during recovery, Use cleaner terminology around truncation and recovery to avoid ambiguity introduced by phrases like relying on high watermark alone

Detailed Feedback:

This is a very strong answer. The candidate correctly defines leader epoch in Kafka as a monotonically increasing value representing successive leadership generations for a partition, which is the core definition the question asked for. They also clearly explain why it is needed: to prevent log divergence and potential data loss during leader changes and broker recovery, especially in cases where relying only on high watermark-based truncation was insufficient. The explanation about a recovering broker consulting the leader using epoch information to determine the correct truncation point is technically solid and shows real understanding beyond a memorized definition. There are a few transcription artefacts (for example, phrases like "HKCs," "log transition," and "silent message calls"), but the intended meaning is still clear. Interpreting through the transcription noise, the candidate is describing the epoch-based truncation/reconciliation mechanism used to keep replicas consistent after failover. That is accurate and directly relevant. What keeps this from an even higher score is that the definition could have been stated a bit more cleanly and explicitly as metadata attached to partition leadership changes, and the explanation of why it is needed could have more directly tied leader epoch to identifying the correct end offset for a given leadership era. But for the scope of the question—define it and explain why it's needed—this is comprehensive and technically correct.

Q23: What must be true for a Kafka Streams application to achieve exactly-once processing (EOS)?

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: hard | Type: OBJECTIVE

Candidate's Verbal Answer:

It must use Kafka transactions to atomically write output records and commit consumer offsets together

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'It must use Kafka transactions to atomically write output records and commit consumer offsets together' is correct.

Q24: Write Java code using a ConsumerRebalanceListener to commit offsets on partitions revoked before a rebalance completes.

Category: Kafka Producer & Consumer | Question Category: Kafka_Upskilling | Difficulty: hard | Type: CODE

Candidate's Code Answer:

```
import org.apache.kafka.clients.consumer.*;
import org.apache.kafka.common.TopicPartition;
import java.time.Duration;
import java.util.*;

public class CompactRebalanceListener{
    public static void main(String args[]){
        Properties props=new Properties();
        props.putAll(Map.of("bootstrap.servers","localhost:9092",
            "groupd.id","compact-group",
            "enable.auto.commit","false",
            "key.deserializer","org.apache.kafka.common.serialization.StringDeserializer",
            "value.deserializer","org.apache.kafka.common.serialization.StringDeserializer"));
```

```

KafkaConsumer<String,String> consumer=new KafkaConsumer<>(props);

Map<TopicPartition,OffsetAndMetadata> offsets=new HashMap<>();

consumer.subscribe(List.of("my-topic"),new CompactRebalanceListener(){
Map<TopicPartition,OffsetAndMetadata> revoked=new HashMap<>(offsets);
revoked.keySet().retainAll(partitions);
if(!revoked.isEmpty())
consumer.commitSync(revoked);
}
});
try(consumer){
while(true){
for(var record:consumer.poll(Duration.ofMillis(100))){
offsets.put(new TopicPartition(record.topic(),record.partition()),new
OffsetAndMetadata(record.offsets + 1));
}
consumer.commitAsync(offsets,null);
}
}
}
}

```

Score: 12/100

Good: Disables auto-commit, which is appropriate for manual offset management, Attempts to track offsets in a map keyed by TopicPartition, Recognizes that revoked partitions should be committed separately during rebalance

Improve: Implement `ConsumerRebalanceListener` instead of instantiating the enclosing class in `subscribe`, Fix Kafka imports and configuration keys, Use the correct `TopicPartition` and `OffsetAndMetadata` construction, Implement `onPartitionsRevoked(Collection)` and commit only revoked offsets there, Fix all syntax and scope errors around listener creation and subscription, Use `record.offset()` and store `new OffsetAndMetadata(record.offset() + 1)` as the map value

Detailed Feedback:

This does not correctly solve the problem and, as written, it does not compile. The question specifically asks for Java code using a ConsumerRebalanceListener to commit offsets on partition revocation before rebalance completes. Your submission attempts that idea, but the implementation is fundamentally broken. Major issues: 1. Wrong import: `org.apache.kafka.common.TopicPartition` is misspelled. It should be `common`, not `commom`. 2. `consumer.subscribe(..., new CompactRebalanceListener(){ ... })` is invalid. `subscribe` expects a `ConsumerRebalanceListener`, not an instance of your enclosing class. 3. The anonymous block inside `new CompactRebalanceListener(){ ... }` is not implementing `onPartitionsRevoked` / `onPartitionsAssigned`. There is no listener implementation here. 4. `partitions` is referenced but never declared in scope. In a proper listener, it would be the parameter to `onPartitionsRevoked(Collection partitions)`. 5. Syntax is broken: braces and parentheses do not match around `subscribe(...)` and the anonymous class block. 6. `new TopicPartition(record.topic(),record.partition(),new OffsetAndMetadata(record.offsets + 1))` is invalid. `TopicPartition` takes only `(String topic, int partition)`. `OffsetAndMetadata` belongs as the map value, not a constructor argument. 7. `record.offsets` does not exist. It should be `record.offset()`. 8. Property key `groupd.id` is wrong; it must be `group.id`. As written, the consumer group configuration is broken. 9. Even ignoring compile errors, the rebalance logic is incomplete. A proper solution should implement `ConsumerRebalanceListener`, commit only revoked partitions in `onPartitionsRevoked`, and usually handle assignment as well. 10. `commitAsync(offsets, null)` on every poll loop is not inherently wrong, but it is not the core requirement and can race with rebalance handling if not carefully managed. Correct approach: create a `ConsumerRebalanceListener` implementation, keep a shared `Map` of processed offsets, and in `onPartitionsRevoked`, filter that map to the revoked partitions and call `consumer.commitSync(revokedOffsets)` before rebalance completes. ■■ Note: Code did not compile in the execution environment. This may be due to missing framework dependencies (e.g. Spring, JUnit) not available in the sandbox runner. Code logic was evaluated by AI review. Compiler output: Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation processing is enabled explicitly (-proc:only, -proc:full). Use -Xlint:options to suppress this message. Use -proc:none to disable annotation processing. CompactRebalanceListener.java:21: error: expected revoked.keySet().retainAll(partitions); ^


```
CompactRebalanceListener.java:21: error: ';' expected revoked.keySet().retainAll(partitions); ^
CompactRebalanceListener.java:22: error: illegal start of type if(!revoked.isEmpty()) ^
CompactRebalanceListener.java:22: error: expected if(!revoked.isEmpty()) ^ CompactRebalanceListener.java:22:
error: ';' expected if(!revoked.isEmpty()) ^ CompactRebalanceListener.java:24: error: ')' or ';' expected } ^
CompactRebalanceListener.java:26: error: illegal start of type try(consumer){ ^ CompactRebalanceListener.java:26:
error: expected try(consumer){ ^ 8 errors
```

Q25: Why is Kafka described as a "distributed commit log"?

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: medium | Type: OBJECTIVE

Candidate's Verbal Answer:

Because each partition is an ordered, append-only log replicated across multiple brokers

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Because each partition is an ordered, append-only log replicated across multiple brokers' is correct.

Q26: How does Kafka handle broker failures in a distributed messaging setup?

Category: Distributed messaging system | Question Category: Kafka_Upskilling | Difficulty: hard | Type: OBJECTIVE

Candidate's Verbal Answer:

The controller reassigns partition leadership to an in-sync follower on another broker

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'The controller reassigns partition leadership to an in-sync follower on another broker' is correct.

Q27: What does "at-least-once" delivery semantics guarantee?

Category: Semantics & Kafka connect | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

Every message is delivered at least once, but duplicates are possible

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'Every message is delivered at least once, but duplicates are possible' is correct.

Q28: What does "broker" mean in Kafka terminology?

Category: Fundamentals & Architecture | Question Category: Kafka_Upskilling | Difficulty: easy | Type: OBJECTIVE

Candidate's Verbal Answer:

A single Kafka server instance that stores data and serves client requests

Score: 100/100

Good: Correct answer selected

Detailed Feedback:

Correct! Your answer 'A single Kafka server instance that stores data and serves client requests' is correct.

This report was generated by the Evalo.
© 2026 IRIS Software. All rights reserved.